Multiversionarea

Seminar 4

Multiversionarea

- Într-un sistem de gestiune a bazelor de date cu multiversionare, fiecare **scriere** a unei înregistrări **X** produce o nouă versiune (copie) a lui **X**
- La fiecare **citire** a lui **X**, sistemul de gestiune a bazelor de date alege una din versiunile lui **X** pentru citire
- Deoarece **NU** există suprapunere între operațiile de scriere, iar operațiile de citire pot citi orice versiune, sistemul de gestiune a bazelor de date are mai multă **flexibilitate** în controlul ordinii scrierilor și citirilor

SQL Server - Versionare la nivel de înregistrare (RLV)

- La **SQL Server** a fost introdusă în versiune **2005**
- Este utilă când este nevoie de date comise, dar nu neapărat de cea mai nouă versiune (încă necomise)
- **Read Committed Snapshot** Isolation și **Full Snapshot** Isolation
  - Cititorul nu blochează niciodată
  - Cititorul primește valoarea comisă anterior
- Toate versiunile mai **vechi** sunt stocate în baza de date ***tempdb***
- Pe baza versiunilor mai **vechi** se poate construi un **snapshot** al datelor

SQL Server - Read Committed Snapshot Isolation

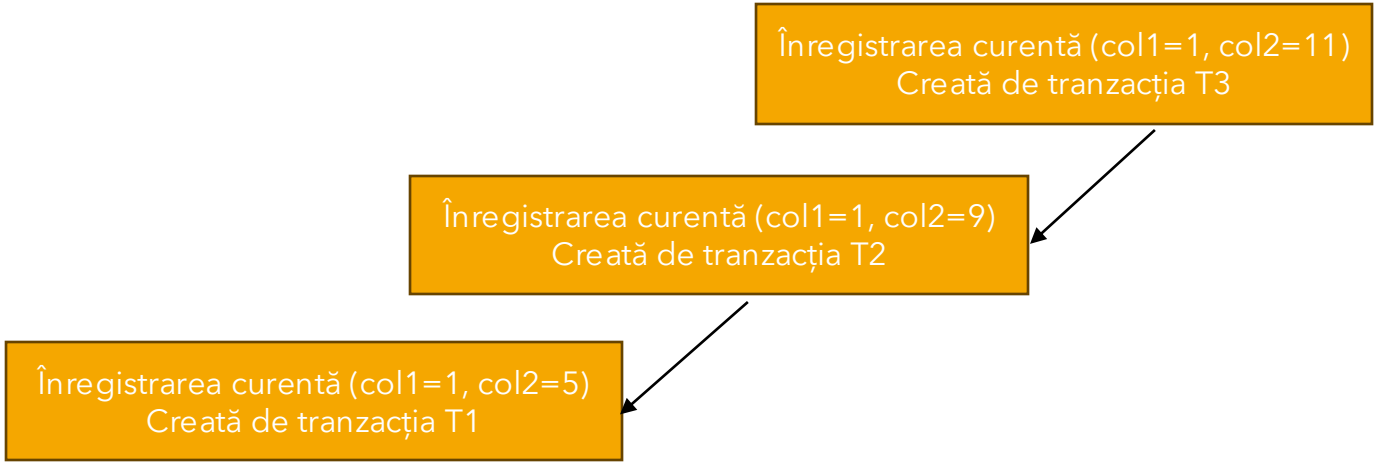
- Toate operațiile văd înregistrările **comise** la **începerea** execuției **lor** =>
  - Snapshot al datelor la nivel de **comandă**
  - Citire **consistentă** la nivel de **comandă**
  - Disponibil la utilizarea nivelului de izolare READ COMMITTED (default) cu opțiunea **READ_COMMITTED_SNAPSHOT** setată pe ON
  - Setarea opțiunii **READ_COMMITTED_SNAPSHOT** pe ON:
    - ALTER DATABASE database_name
    - SET READ_COMMITTED_SNAPSHOT ON;

SQL Server - Full Snapshot Isolation

- Toate operațiile văd înregistrările **comise** la **începerea** execuției **tranzacției** =>
  - *Snapshot* al datelor la nivel de **tranzacție**
  - Citire **consistentă** la nivel de **tranzacție**
  - Nivelul de izolare **SNAPSHOT** poate fi folosit dacă opțiunea **ALLOW_SNAPSHOT_ISOLATION** este setată pe ON
  - Setarea opțiunii ALLOW_SNAPSHOT_ISOLATION pe ON:
    - ALTER DATABASE database_name
    - SET ALLOW_SNAPSHOT_ISOLATION ON;

Versionarea la nivel de înregistrare

- O înregistrare conține TSN (transaction sequence number)
- Toate versiunile sunt stocate într-o listă înlănțuită



Versionarea la nivel de înregistrare

- **Avantaje:**
  - Nivelul de concurență este mai ridicat
  - Mărește performanța triggerelor și a creării indecșilor
- **Dezavantaje:**
  - Cerințe de gestiune suplimentare pentru monitorizarea utilizării tempdb
  - Performanța mai scăzută a operațiilor de modificare
  - Viteza cititorilor este afectată de costul traversării listelor înlănțuite
  - Se rezolvă conflictul dintre cititori și scriitori, dar tot nu sunt permiși scriitorii simultani

Crearea indecșilor și versionarea la nivel de înregistrare

- În versiunile **anterioare SQL Server 2005**, operațiile pe indecși erau *offline*
- **Index clustered** → tabel blocat exclusiv
- **Index nonclustered** → tabel disponibil doar pentru citire
- Din SQL Server 2005, indecșii pot fi creați/reconstruiți **online** (ONLINE = ON) (doar Ent edition, Std in v2019)
- Operațiile online permit acces concurent la date
- **Row versioning** poate fi utilizată pentru a asigura citiri consistente și a reduce blocările

Niveluri de izolare și anomalii de concurență - SQL Server

Denumire	Dirty Reads	Unrepeatable Reads	Phantom Reads	Update conflict	Model de concurență
Read Uncommitted	DA	DA	DA	NU	Pesimit
Read Committed Locking	NU	DA	DA	NU	Pesimit
Read Committed Snapshot	NU	DA	DA	NU	Optimist
Repeatable Read	NU	NU	DA	NU	Pesimit
Snapshot	NU	NU	NU	DA	Optimist
Serializable	NU	NU	NU	NU	Pesimist

Versionare PostgreSQL

- PostgreSQL folosește **MVCC (Multi-Version Concurrency Control)** pentru acces concurent
- Regulă de bază: **"Readers never block writers, writers never block readers" -**Operațiile SELECT nu blochează INSERT, UPDATE sau DELETE
- Fiecare tranzacție lucrează pe un **snapshot logic** al bazei de date
- Concurența este gestionată **fără locking intensiv**
- Scopul principal: **citiri consistente + performanță ridicată**

Versionare PostgreSQL

- Avantaje:
  - Concurență ridicată - citirile și scrierile nu se blochează reciproc
  - Predictibilitate - fiecare tranzacție vede o imagine consistentă a datelor
  - Scalabilitate - performanță bună în scenarii cu multe read-uri concurente
- Dezavantaje:
  - Dead records
  - VACUUM (maintenance)

Versionare PostgreSQL - implementare

- Fiecare rând (tuplă) conține câmpuri ascunse: xmin (ID-ul tranzacției care a creat rândul) și xmax (ID-ul tranzacției care l-a șters/actualizat).
  - **UPDATE = DELETE + INSERT:** O actualizare nu suprascrie datele fizice, ci creează un nou rând cu un xmin nou, în timp ce cel vechi primește un xmax.
  - **Vizibilitatea:** O tranzacție vede un rând doar dacă:
    - Tranzacția care l-a creat a făcut COMMIT.
    - Tranzacția care l-a șters (dacă există) nu a făcut încă COMMIT.
  - **Dead Tuples:** Versiunile vechi ale rândurilor care nu mai sunt vizibile niciunei tranzacții active devin "reziduuri" (bloat) > **Nevoia de mentenanță**
  - **Rolul VACUUM:** Scanează tabelele și marchează spațiul ocupat de rândurile moarte ca fiind "liber" pentru viitoare inserări.
  - **Autovacuum:** Un proces de fundal (daemon) care rulează automat când pragul de modificări într-o tabelă este depășit.
  - **VACUUM FULL:** O variantă agresivă care rescrie tabela complet pentru a recupera spațiul pe disc (necesită lock exclusiv, deci oprește accesul la tabelă).

Niveluri de izolare și anomalii de concurență - PostgreSQL

Denumire	Dirty Reads	Unrepeatable Reads	Phantom Reads	Serialization anomaly	Model de concurență
Read Uncommitted	NU*	DA	DA	DA	Optimist
Read Committed	NU	DA	DA	DA	Optimist
Repeatable Read	NU	NU	NU**	DA	Optimist
Serializable	NU	NU	NU	NU	Optimist

- * Dirty Reads – NU sunt posibile în PostgreSQL
- ** Phantom Reads – NU apar în REPEATABLE READ – deoarece folosește Snapshot Isolation

	PostgreSQL	Oracle	SQL Server
Versionare înregistrare	Implicit, versiuni în heap	Implicit, versiuni în undo tablespace	Opțional, versiuni în tempdb
Overhead MVCC	Bloat în tabele + VACUUM periodic	Presiune pe **undo tablespace**	Presiune pe **tempdb** (când e activat)
Lock-uri explicite	Opționale (mai rare in RR)	Similar cu PG, row locks doar pt. writes	**Esențiale** în RC
Niveluri de izolare	4 (dar RU=RC)	Doar 2 (+RO)	5 complete
REPEATABLE READ	Da (cu SI)	NU	DA (lock-based)
SERIALIZABLE	True SSI	SI (Snapshot Isolation) - not true	True (lock-based)
Ph Reads în RR	**NU**	**N/A**	**DA**
Dirty Reads	Niciodată	Niciodată	DA (în RU)
MVCC default	DA	DA	NU

Comparație PostgreSQL/Oracle/SQL Server

Probleme de concurență - Setup

PostgreSQL

```
CREATE TABLE Movies (
movie_id SERIAL PRIMARY KEY,
title VARCHAR(100),
year INT);

CREATE TABLE Actors (
actor_id SERIAL PRIMARY KEY,
firstname VARCHAR(100),
lastname VARCHAR(100));
```

MS SQL Server

```
CREATE TABLE Movies (
movie_id INT PRIMARY KEY IDENTITY,
title VARCHAR(100),
year INT);

CREATE TABLE Actors (
actor_id INT PRIMARY KEY IDENTITY,
firstname VARCHAR(100),
lastname VARCHAR(100));
```

```
INSERT INTO Movies (title, year) VALUES ('IT',2017), ('Red Sparrow',2018);
INSERT INTO Actors (firstname, lastname) VALUES ('Bill', 'Skarsgard');
```

Concurență - Pesimist vs Optimist

- Deschidem două ferestre noi de interogare (două conexiuni)
- În prima fereastră de interogare punem următoarea tranzacție:

```
BEGIN;
UPDATE movies SET year=2020 WHERE title='IT';
SELECT pg_sleep(6);    -- SQL SERVBER: WAITFOR DELAY '00:00:06';
SELECT title, year, clock_timestamp() FROM movies WHERE title='IT'; -- SQL Server: GETDATE()
COMMIT;
```

Concurență - Pesimist vs Optimist

- În a doua fereastră de interogare punem următoarea tranzacție:

```
BEGIN;
SELECT pg_sleep(3);    -- SQL Server: WAITFOR DELAY '00:00:03';
SELECT title, year, clock_timestamp() FROM movies WHERE title='IT'; -- SQL Server: GETDATE()
COMMIT;
```

- Începem execuția primei și a celei de a doua tranzacții (în această ordine)

Concurență - Pesimist vs Optimist

- În prima fereastră de interogare putem vedea următorul rezultat:

Results | Meta data | Info | Overview / Charts | Rotated table

title	year	clock_timestamp
IT	2020	2026-02-07 19:20:22.168791

- În cea de-a doua ferestră vedem:

Results | Meta data | Info | Overview / Charts | Rotated table

title	year	clock_timestamp
IT	2017	2026-02-07 19:20:16.362545

- După cum se poate observa citirea din a doua fereastră a fost executată prima, inainte citirii si comiterii tranzacției din primul ecran. În modul pesimist (default în SQL Server), citirea din al doilea ecran ar fi fost blocată și ar fi așteptat terminarea primei tranzacții

Probleme de Concurență - Conflict de update

- Deschidem două ferestre noi de interogare (două conexiuni)
- În prima fereastră de interogare punem următoarea tranzacție:

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ; -- SNAPSHOT
BEGIN;
SELECT * FROM movies; -- foarte important – snapshotul se ia la prima comanda SQL executată
SELECT pg_sleep(3);    -- WAITFOR DELAY '00:00:03';
UPDATE movies SET title='The Snowman' WHERE year=2017;
COMMIT;
```

Probleme de Concurență - Conflict de update

- În a doua fereastră de interogare punem următoarea tranzacție:

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ; -- SNAPSHOT
BEGIN;
UPDATE movies SET title='Thor Ragnarok' WHERE year=2017;
SELECT pg_sleep(3);    -- SQL Server: WAITFOR DELAY '00:00:03';
COMMIT;
```

- Începem execuția primei și a celei de a doua tranzacții (în această ordine)

Probleme de Concurență - Conflict de update

- În prima fereastră de interogare putem vedea următorul rezultat:

```
Error occurred in: COMMIT
Error: ERROR: could not serialize access due to concurrent update
SQLState: 40001
ErrorCode: 0
Error occurred in:
UPDATE movies SET title='The Snowman' WHERE year=2017
```

- În cea de-a doua fereastră update-ul s-a efectuat cu succes:

movie_id	title	year
2	Red Sparrow	2018
1	Thor Ragnarok	2017

- Eroarea apare la execuția script-ului din primul ecran deoarece o tranzacție de tip Snapshot garanteză ca datele odată citite nu se schimbă pe parcursul tranzacției.

Locking pesimist într-un sistem cu versionare

- În mod normal un SELECT nu blocheaza un UPDATE
- Există scenarii unde totuși dorim blocarea unei înregistrări până la finalizarea tranzacției (plată, rezervare loc)
- **Efect:** Pune un lock de tip **Exclusive** pe rândurile returnate.
- **Comportament:** Orice altă sesiune care încearcă să facă **UPDATE, DELETE sau SELECT FOR UPDATE** pe aceleași **rânduri** va fi pusă în așteptare (Queue). SELECT simplu merge în continuare fără locking/așteptare.
- **Utilitate:** Previne fenomenul de "Lost Update" fără a ridica nivelul de izolare la Serializable.

Exemplu – SELECT FOR UPDATE

- Deschidem două ferestre noi de interogare (două conexiuni)
- În prima fereastră de interogare punem următoarea tranzacție:

```
BEGIN;
SELECT title FROM movies WHERE year=2017 FOR UPDATE;
SELECT pg_sleep(6);    -- WAITFOR DELAY '00:00:06';
UPDATE movies SET year=2018 WHERE year=2017;
COMMIT;
```

Exemplu – SELECT FOR UPDATE

- În a doua fereastră de interogare punem următoarea tranzacție:

```
BEGIN;
SELECT title FROM movies WHERE year=2017 FOR UPDATE;
UPDATE movies SET year=2020 WHERE year=2017;
COMMIT;
```

- Începem execuția primei și a celei de a doua tranzacții (în această ordine)

Exemplu – SELECT FOR UPDATE

- În prima fereastră de interogare putem vedea următorul rezultat:

movie_id	title	year
2	Red Sparrow	2018
1	Thor Ragnarok	2018

- În cea de-a doua ferestră tranzacția s-a încheiat cu succes, dar nu a actualizat nicio o înregistrare, deoarece instrucțiunile de SELECT FOR UPDATE a aștetat după comiterea primei tranzacții, iar apoi nu a mai găsit nicio înregistrare de actualizat

Monitorizarea blocărilor în SQL Server

- SQL Server Extended Events
- Procedura stocată sistem sp_lock
- Dynamic management view-ul sistem sys.dm_tran_locks
- Dynamic management view-ul sistem sys.dm_tran_active_transactions
- **DBCC LOG - informații despre logul de tranzacții**

Resurse și referințe

- **SQL Server:**
  - https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?view=sql-server-ver16
  - https://learn.microsoft.com/en-us/sql/relational-databases/extended-events/quick-start-extended-events-in-sql-server?view=sql-server-ver16
  - https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-lock-transact-sql?view=sql-server-ver16
  - https://learn.microsoft.com/en-us/sql/relational-databases/system-dynamic-management-views/sys-dm-tran-locks-transact-sql?view=sql-server-ver17
- **PostgreSQL:**
  - https://www.postgresql.org/docs/18/mvcc.html